

CS 4150 ALGORITHMS

Spring 2026

Assignment 3: Prune and Search

Due Date: 11:59 p.m., Tuesday, February 24, 2026

1. **(20 points)** Suppose that you are given an array $A[1 \cdots n]$ of n entries, with each entry having a distinct number (that is, no two numbers of A are equal). You are told that the sequence of values $A[1], A[2], \dots, A[n]$ is *unimodal*: For some index p between 1 and n , the values in the array entries increase up to position p in A and then decrease the rest of the way until position n . (So if you were to draw a plot with the array index i on the x -axis and the value of the entry $A[i]$ on the y -axis, the plotted points would rise until x -coordinate p , where they would achieve their maximum, and then fall from there on.)

You would like to find the “peak entry” p without having to read the entire array – in fact, by reading as few entries of A as possible. Show how to find the entry p by reading at most $O(\log n)$ entries of A . In other words, design an $O(\log n)$ time algorithm to find the peak entry p .

For example, let $A = \{1, 4, 6, 8, 11, 12, 10, 9, 7\}$, which is a unimodal array. The peak entry of A is 12. So the output of your algorithm may be either 12 or the index of 12 in A .

2. **(20 points)** In the SELECTION algorithm we studied in class, the input numbers are divided into groups of five elements each. Will the algorithm still work in $O(n)$ time if they are divided into groups of **seven** elements each? Please justify your answer (i.e., explain why your answer is correct and provide the analysis; please give your recurrence and solve the recurrence).

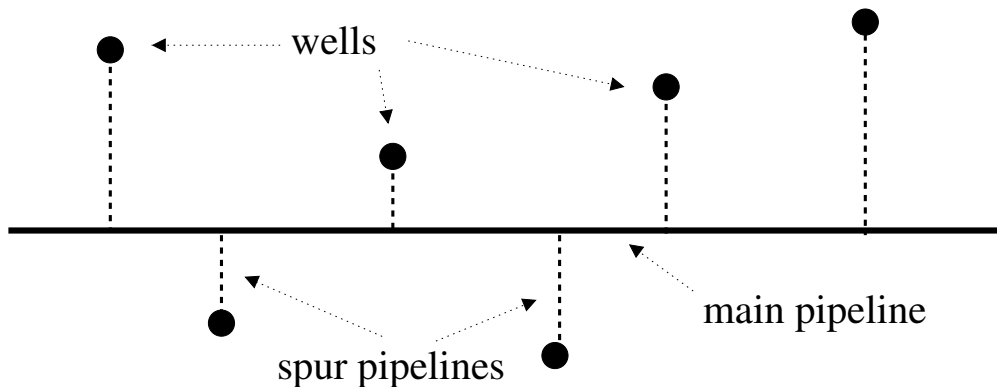


Figure 1: Illustrating Problem 3: the horizontal solid line is the main pipeline and the dashed vertical segments are the spur pipelines.

3. **(20 points)** Suppose that you are consulting for an oil company, which is planning a large pipeline (called the *main pipeline*) running horizontally from east to west through an oil field of n wells. From each well, a *spur pipeline* is to be connected directly to the main pipeline along a shortest path (going to either the north or the south), as shown in Figure 1.

Suppose that there are n wells, represented by n points $p_1, p_2, \dots, p_n$ in the plane. We are given the x - and y -coordinates of the n wells $p_i = (x_i, y_i)$ for $i = 1, 2, \dots, n$. Note that the wells are not given in any sorted order. Our goal is to pick an optimal location for the main pipeline (i.e., find the y -coordinate of the main pipeline) such that the **total sum of the lengths** of the spur pipelines is minimized. Essentially, we are looking for a y -coordinate value y such that the sum $\sum_{1 \leq i \leq n} |y - y_i|$ is minimized, and the value y that minimizes the sum is the optimal location for the main pipeline.

For simplicity, we assume that no two wells have the same x -coordinate or y -coordinate.

Design an $O(n)$ time algorithm to compute an optimal location for the main pipeline.

4. **(20 points)** We now consider a *weighted* version of Problem 3. Because of the locations, the cost for building spur pipelines on different wells may be different. For each well p_i , it is associated with a *weight* $w_i > 0$, representing the cost for building a spur line of a unit length. In other words, if the length of the spur line for p_i is r , then the cost for building the spur line is $w_i \cdot r$.

For each well p_i , we are given its x - and y -coordinates (x_i, y_i) and also its weight w_i . Our goal is to pick an optimal location for the main pipeline (i.e., find the y -coordinate of the main pipeline) such that the **total sum of the cost** of building the spur pipelines for all wells is minimized. Essentially, we are looking for a y -coordinate value y such that the sum $\sum_{1 \leq i \leq n} (w_i \cdot |y - y_i|)$ is minimized, and the value y that minimizes the sum is the optimal location for the main pipeline.

Again, we assume that no two wells have the same x -coordinate or y -coordinate.

Design an $O(n)$ time algorithm to compute an optimal location for the main pipeline.

5. **(20 points)** Here is a generalized version of the selection problem, called *multiple selection*. Let $A[1 \dots n]$ be an array of n numbers. Given a sequence of m sorted integers $k_1, k_2, \dots, k_m$, with $1 \leq k_1 < k_2 < \dots < k_m \leq n$, the *multiple selection problem* is to find the k_i -th smallest number in A for all $i = 1, 2, \dots, m$. For simplicity, we assume that no two numbers of A are equal. Note that m is no greater than n by definition.

For example, let $A = \{1, 5, 9, 3, 7, 12, 15, 8, 21\}$, and $m = 3$ with $k_1 = 2$, $k_2 = 5$, and $k_3 = 7$. Hence, the goal is to find the 2nd, the 5-th, and the 7-th smallest numbers of A , which are 3, 8, and 12, respectively.

Design an $O(n \log m)$ time algorithm for the problem.

Note: Since $m \leq n$, $O(n \log m)$ is better than $O(n \log n)$ especially when m is much smaller than n (e.g., when $m \leq \log n$). Therefore, if your algorithm runs in $O(n \log n)$ time, then it would NOT be sufficient. This further implies that you should not attempt to sort A because that would cost $O(n \log n)$ time.

Hint: Use the linear-time selection algorithm studied in class and the divide-and-conquer technique.

Total Points: 100